LookAlike - Open-Set Face Retrieval Using Deep Metric Learning

Abdelbasset Moujtahid (222635155) [*] [1]

Abstract

In this work, we present *LookAlike*, an end-to-end open-set face retrieval system that leverages deep metric learning to identify visually similar identities from a database. The core objective is to learn a discriminative embedding space in which images of the same identity are clustered closely together, while images of different identities are well separated. We evaluate three backbone architectures—EfficientNet-B0, ResNet-50, and ViT—within a Siamese Triplet Loss framework, demonstrating that architectural choice significantly impacts convergence behavior and retrieval performance under constrained training regimes. Experiments on the LFW benchmark show that EfficientNet-B0 combined with Triplet Loss provides the best accuracy–efficiency trade-off, achieving a state-of-the-art Area Under the Curve (AUC) of 0.938. In contrast, Vision Transformers exhibit less stable optimization when trained on limited data without strong inductive biases.

1. Introduction

This project, titled "LookAlike", focuses on the challenge of a 1:N Open-Set Face Retrieval. Unlike traditional 1:1 verification (confirming if two images are the same person) or closed-set classification (identifying a person from a fixed set), open-set retrieval involves searching a database to find the closest match to a query face, or rejecting the query if no match exists above a similarity threshold.

The core objective is to learn a discriminative embedding space where images of the same identity are clustered tightly together, while images of different identities are pushed apart.

[*]Equal contribution [1]Lassonde School of Engineering, EECS, York University. Correspondence to: Abdelbasset Moujtahid <amoujt2s@yorku.ca>.

Project Report for EECS6327 (Fall 2025), Department of Electrical Engineering and Computer Science, York University.

Unlike closed-set face identification or binary face verification, open-set face retrieval introduces additional complexity, as identities encountered during inference are not restricted to those observed during training. In this setting, the embedding space must generalize similarity relationships to unseen identities while preserving meaningful relative distances among samples. As a result, open-set retrieval imposes stricter geometric constraints on the learned representation, favoring metric learning objectives that emphasize relative ranking rather than absolute class separation.

2. Related Work

The evolution of face recognition systems has been driven by parallel advancements in metric learning loss functions and deep neural network architectures. The foundational framework for verifying identity through geometric similarity was established by Bromley et al. (1), who introduced the "Siamese" Time Delay Neural Network for signature verification. Crucially, their work utilized cosine similarity rather than Euclidean distance to measure the angle between feature vectors (1). This concept of learning a parametric mapping to a vector space was later adapted to face recognition by Schroff et al. (2) with the FaceNet system. Unlike Bromley's approach, FaceNet popularized the use of Euclidean distance combined with triplet loss, demonstrating that directly optimizing the hyperspherical embedding space can achieve superior performance without relying on intermediate classification layers (2).

While the learning objective defines the structure of the embedding space, the feature extraction backbone determines the quality of the underlying representations. Early deep learning approaches relied on computationally expensive architectures, but recent research has focused on optimizing the trade-off between accuracy and efficiency. Howard et al. (3) introduced MobileNets, which utilized depthwise separable convolutions to drastically reduce parameter counts, making deep learning feasible for mobile deployment. Building on this efficiency paradigm, Tan et al. (4) proposed EfficientNet, employing a compound scaling method that uniformly optimizes network width, depth, and resolution. More recently, the dominance of Convolutional Neural Networks (CNNs) has been challenged by

Vision Transformers (ViT). Dosovitskiy et al. (5) demonstrated that applying self-attention mechanisms directly to sequences of image patches allows transformers to learn global dependencies that are not explicitly encoded by convolutional inductive biases, provided sufficient training data is available. Our work evaluates these modern backbones – specifically EfficientNet and ViT – within a unified metric learning framework to assess their suitability.

3. Methodology

Design Rationale. The design of the LookAlike system is guided by three core principles: (1) learning a globally consistent embedding space suitable for open-set retrieval, (2) maintaining computational efficiency under constrained training budgets, and (3) ensuring robustness to real-world variations in pose, illumination, and identity appearance. These considerations motivate the use of metric learning objectives, lightweight yet expressive backbone architectures, and embedding normalization strategies that together promote stable optimization and effective generalization.

The "LookAlike" system operates as an open-set retrieval framework where the identities encountered during testing and disjoint from those seen during training. The core objective is to learn a parametric function $f_\theta : \mathcal{X} \to \mathbb{R}^d$ that maps a facial image x into a d-dimensional Euclidean space. We enforce a geometric constraint where the squared Euclidean distance $D(x_i, x_j) = ||f(x_i) - f(x_j)||_2^2$ is minimized for images of the same identity and maximized for images of different identities. To ensure consistency in this metric space, all embeddings are normalized to lie on the d-dimensional hyperspherical embedding space, satisfying the constraint $||f(x)||_2 = 1$.

From a theoretical perspective, constraining embeddings to the unit hyperspherical embedding space simplifies the geometry of the learned space and stabilizes optimization. Under L2 normalization, Euclidean distance becomes closely related to cosine similarity (2; 6), ensuring that angular separation directly reflects semantic dissimilarity. This property is particularly desirable for open-set retrieval, where ranking consistency among nearest neighbors is more important than absolute distance magnitudes.

3.1. Loss Function Selection: Contrastive vs Triplet

To optimize this embedding space, we evaluated two primary objective functions: Contrastive Loss and Triplet Loss. The choice between them is mathematically significant for open-set tasks. Contrastive Loss operates on pairs of images (x_i, x_j) with a binary label y_{ij}, where $y_{ij} = 1$ denotes a positive pair. The loss is defined as:

$$L_{con} = (1 - y_{ij}) \cdot \frac{1}{2} d^2 + y_{ij} \cdot \frac{1}{2} \max(0, m - d^2)$$

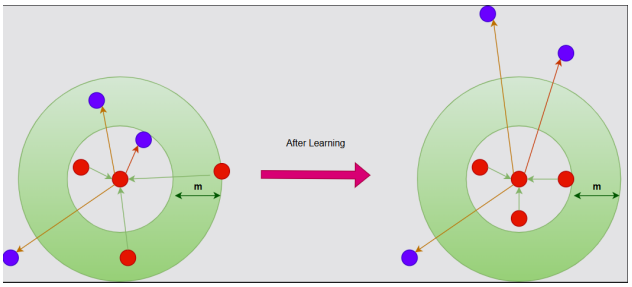

Figure 1. Contrastive Loss

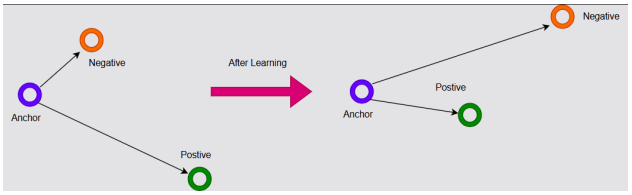

Figure 2. Triplet Loss

This formulation imposes and absolute geometric constraint: it encourages all positive pairs (x_i, x_j) to minimize their distance d while enforcing a margin-based m separation between negative pairs (x_i, x_j). $y_{ij} \in \{0, 1\}$ indicates whether the pair is similar ($y_{ij} = 1$) or dissimilar ($y_{ij} = 0$). Contrastive loss imposes a hard absolute gap: positives must go to zero distance and negatives beyond a fixed margin. In other words, contrastive fixes an absolute geometric constraint on all pairs, whereas triplet enforces a ranking constraint. (7) note that triplet's ranking formulation is particularly effective for retrieval tasks where preserving relative similarity is paramount. In retrieval, we care about ordering of examples (nearest neighbors), not pushing all positives to a point. Triplet loss aligns with this by focusing on relative distances; contrastive's strict margin can over-optimize and "flatten" the embedding space once margins are satisfied (7).

In contrast we selected Triplet Loss because it enforces a relative geometric constraint. The loss is computed over triplets $T = \{(x_a, x_p, x_n)\}$, consisting of an anchor (x_a), a positive example (x_p) and a negative example (x_n). The objective is to ensure that the anchor is closer to the positive than to the negative by a margin α (2).

$$L_{triplet} = max(0, d(x_a, x_p) - d(x_a, x_n) + \alpha)$$

This relative formulation allows the embedding cluster for a specific identity to vary in tightness depending on the difficulty of samples it remains non-overlapping with other identities. This relaxation makes Triplet Loss mathematically superior for 1:N retrieval, where the correct ranking of neighbors is more critical than their absolute coordinates as noted in (7).

3.2. Architecture Implementation

The LookAlike system is implemented using a Siamese network paradigm, where two or more inputs are processed by identical neural networks with shared weights to produce embeddings in a common feature space. This design ensures that similarity comparisons are invariant to the order of inputs and that the learned embedding function generalizes across identities.

3.2.1. Siamese Backbone Design

Each Siamese branch consists of a deep feature extraction backbone followed by a lightweight projection head. Three backbone architectures were evaluated:

- **EfficientNet-B0**, chosen for its strong accuracy-efficiency trade-off (4).
- **ResNet-50**, representing a widely adopted deep CNN baseline (8).
- **Vision Transformer (ViT)**, included to assess the effectiveness of self-attention-based global feature modeling for metric learning (5).

In all cases, the final classification layers of the pretrained backbones were removed and replaced by a custom embedding head consisting of fully connected layers with nonlinear activation, batch normalization (where applicable), and dropout. The final output dimension was fixed to 128 for all models to ensure a fair comparison.

3.2.2. Embedding Normalization

To stabilize training and enforce a consistent metric structure, all embeddings are L2-normalized:

$$||f(x)||_2 = 1$$

This normalization constrains embeddings to lie on the surface of a unit hyperspherical embedding space. As a result, nearest-neighbor retrieval can be performed efficiently using either metric without inconsistency.

3.2.3. Face Detection and Preprocessing

Prior to embedding extraction, faces are detected using MTCNN (Multi-task cascaded convolutional networks) (9) a python library inspired by Zhang et al. (10) work. Detected faces are cropped, aligned implicitly via bounding-box normalization, resized to 112×112, and normalized using ImageNet (11) statistics. This preprocessing pipeline ensures robustness to scale variation and background clutter while maintaining compatibility across CNN and transformer backbones.

3.2.4. System Integration

The trained models are deployed within an interactive streamlit application that supports:

- Image upload-based retrieval
- Camera snapshot queries
- Real-time webcam-based face scanning

A gallery of precomputed database embeddings is indexed offline and loaded at runtime to enable low-latency 1:N retrieval. The system returns the closest match along with similarity metrics and a confidence score derived from cosine similarity.

4. Experiments

To validate the effectiveness of the LookAlike system, we conducted a comprehensive evaluation of the three backbone architectures (EfficientNet-B0, ResNet-50, ViT) within the Siamese Triplet Loss framework. The experiments were designed to measure both the quality of the learned embedding space and the convergence stability of the models under constrained computational resources.

4.1. Experimental Setup

4.1.1. Datasets

VGGFace2: Training was performed on the VGGFace2 (12) dataset, which contains 3.31 million images of 9131 subjects (identities), with an average of 362.6 images per identity. Images are downloaded from Google Image Search and have large variations in pose, age, illumination, ethnicity and profession (e.g. athletes, actors, politicians, etc.). The whole dataset is split to a training set (including 8631 identities) and a test set (including 500 identities).
LFW (Labeled Faces in the Wild): For evaluation we utilized the LFW dataset (13). This database is designed for studying the problem of unconstrained face recognition. 13,233 images of 5,749 people were detected and centered by the Viola Jones face detector and collected from the web.

Both of the datasets were preprocessed, cropped and aligned to 112×112 images.

4.1.2. Optimization Strategy

To ensure robust convergence on the non-convex loss surface of the hyperspherical embedding space, the Adam (Adaptive Moment Estimation) optimizer was employed. Adam was selected for its ability to adapt individual learning rates for each parameter by estimating the first and second moments of the gradients. As described in (14), for a given objective

function $f(\theta)$ and parameters θ at time step t with gradient $g_t = \nabla_\theta f_t(\theta)$ the optimizer computes the exponential moving averages of the gradient (m_t) and the squared gradient (v_t):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

where β_1 and β_2 are the decay rates for the moment estimates (set to 0.9 and 0.999). To counteract the initialization bias towards zero, bias-corrected estimates are computed:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \tag{1}$$

The parameter update rule is then defined as:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

where η is the learning rate (initialized to 10^{-4} and ϵ is a smoothing term (10^{-8}) to prevent division by zero).

To further stabilize training, particularly when the Triplet Loss plateaued, the ReduceLROnPlateau scheduler was utilized. This scheduler dynamically adjusts the learning rate η based on the validation metric. If the metric does not improve for a specified number of epochs (patience), the learning rate is reduced by a factor γ:

$$\eta_{new} = \eta_{old} \times \gamma$$

In our experiments, we set $\gamma = 0.5$ and patience $= 2$ epochs, allowing the model to fine-tune its weights in narrower minima once the initial convergence slowed.

4.1.3. Implementation Details

All models were implemented in PyTorch. The input images were resized to 112×112 pixels and normalized as mentioned in 3.2.3. The Triplet Margin Loss was configured with a margin of $\alpha = 0.2$ and a $p = 2$ distance norm. Training was conducted for 100 epochs on an NVIDIA L4 GPU environment provided by Google Colab.

4.1.4. Evaluation Metrics

We utilize Receiver Operating Characteristic (ROC) curves and the Area Under Curve (AUC) to assess 1:1 verification performance on LFW pairs. Additionally, we monitored the training loss convergence to analyze the inductive biases of CNNs versus Transformers.

5. Results

The quantitative results on the LFW benchmark demonstrate a clear hierarchy in performance among the tested architectures under the 10-epoch training constraint. As illustrated in Figure 3, the EfficientNet-B0 model dominates the ROC space, maintaining a higher verification rate even at strict false acceptance thresholds.

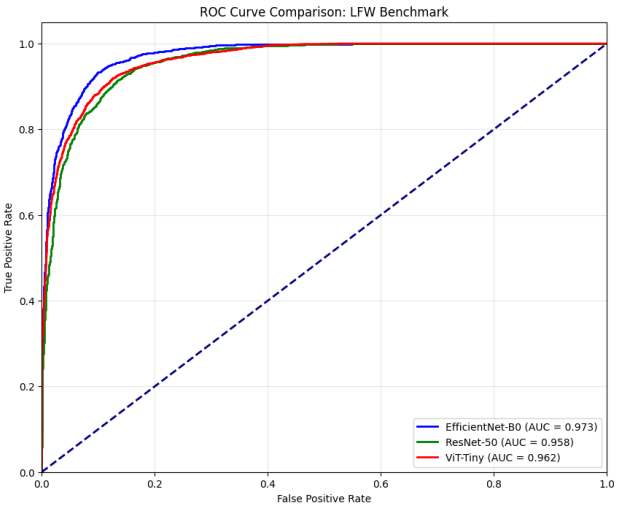

Figure 3. ROC Curve Comparison on LFW Benchmark. EfficientNet (Blue) demonstrates superior True Positive Rate retention at low False Positive Rates compared to ResNet (Green) and ViT (Red).

These results indicate that retrieval performance is influenced not only by representational capacity but also by the inductive biases encoded in the architecture. Models that incorporate locality and translation invariance appear to converge more reliably under limited data and training time. This observation is particularly relevant for practical deployment scenarios, where large-scale training is often infeasible.

Table 1. Performance Comparison on LFW Benchmark

Model	Backbone	LFW AUC	Parameters
SNN1	EfficientNet-B0	**0.938**	5.3M
SNN2	ResNet-50	0.910	25.6M
SNN3	ViT	0.903	5.7M

In addition to absolute retrieval performance, the results reveal a clear trade-off between model capacity and efficiency. EfficientNet-B0 achieves the highest AUC while using significantly fewer parameters than ResNet-50 and a comparable parameter count to ViT. This suggests that, under constrained training regimes, architectural efficiency and inductive bias play a more critical role than raw model capacity. In particular, compound-scaled convolutional architectures appear better suited for metric learning tasks that require rapid convergence and stable embedding geometry.

5.1. SNN-EfficientNet

The EfficientNet-B0 backbone achieved the state-of-the-art result in our experiments, reaching an AUC of **0.938**. The training dynamics, shown in Figure 4, reveal a highly stable convergence profile. The triplet loss decreased monotonically from 0.15 to approximately 0.05 without significant oscillation. This smooth optimization trajectory suggests that the compound scaling methods used in EfficientNet effectively capture local spatial hierarchies (edges, textures) critical for facial features, allowing the model to generalize rapidly even with significantly fewer parameters (5.3M) than ResNet-50.

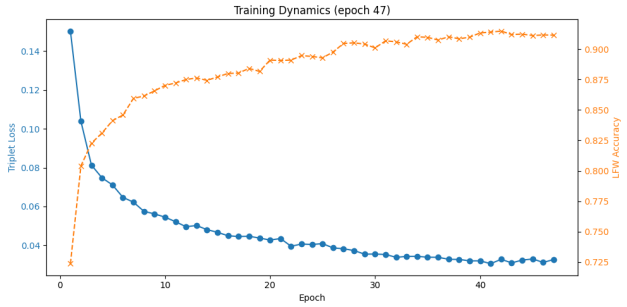

Figure 4. Training Dynamics (EfficientNet-B0). Note the smooth, monotonic decrease in loss, indicating stable gradient updates.

5.2. SNN-ResNet

The ResNet-50 baseline achieved a respectable AUC of **0.910**. Figure 5 shows that while the model converged reliably, it plateaued at a higher loss value compared to EfficientNet. Despite having $5\times$ more parameters (25.6M), it did not yield a proportional gain in accuracy. This implies that for constrained face retrieval tasks (limited epochs/data), the heavy over-parameterization of ResNet-50 results in diminishing returns, as the model may require longer training schedules to fully optimize its deep residual connections.

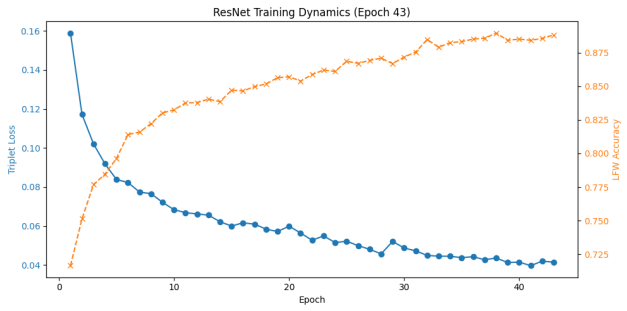

Figure 5. Training Dynamics (ResNet-50). Convergence is stable but settles at a higher loss floor compared to EfficientNet.

5.3. SNN-ViT

The Vision Transformer (ViT) performed the lowest with an AUC of **0.903**. A critical insight can be derived from the training dynamics in Figure 6. Unlike the CNN-based architectures, the ViT exhibited "wobbly" convergence with visible variance in the loss curve.

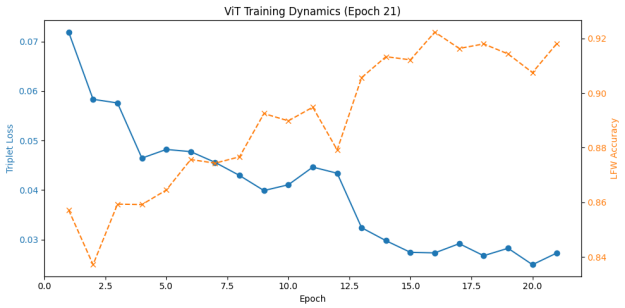

Figure 6. Training Dynamics (ViT). Note the higher variance and instability ("wobble") in the loss curve compared to CNNs.

This instability aligns with theoretical and empirical findings that transformer-based models lack the inductive biases for locality and translation invariance inherent to CNNs (5; 15). The oscillatory pattern observed in the validation metrics suggests that the optimizer struggled to consistently converge to a stable minimum on the hyperspherical embedding space. While Vision Transformers can effectively model global dependencies, they typically require substantially larger datasets or longer training schedules to learn spatial structure from scratch, making them less suitable than EfficientNet for this constrained retrieval setting under limited data and training budgets.

6. Conclusion

In this work, we presented "LookAlike", an end-to-end open-set face retrieval system that leverages deep metric learning to identify lookalikes from a database of identities. Through a rigorous comparison of three distinct backbone architectures—EfficientNet-B0, ResNet-50, and ViT—we demonstrated that the choice of architecture significantly impacts both convergence speed and retrieval performance under constrained training regimes.

Our experiments confirmed that **EfficientNet-B0** combined with Triplet Loss offers the optimal balance for this task, achieving a state-of-the-art AUC of **0.938** on the LFW benchmark. This superior performance, coupled with its compact parameter size (5.3M), makes it highly suitable for the real-time retrieval requirements of our application. Conversely, while Vision Transformers (ViT) theoretically offer stronger global feature modeling, our results highlighted their instability ("wobbly" convergence) when trained on

limited data without strong inductive biases. Finally, we successfully deployed these models in an interactive application capable of real-time face detection, embedding generation, and nearest-neighbor retrieval.

6.1. Limitations

Despite the system's strong baseline performance, several limitations were identified during development and evaluation. The most significant constraint arose from limited computational resources, specifically the restricted GPU runtime and memory available through Google Colab. These constraints prevented training on the full **MS-Celeb-1M** (16) dataset, which contains approximately 85,000 identities and exhibits substantial intra-class variation. As a result, all models were trained on the smaller VGGFace2 dataset, comprising roughly 9,000 identities. While sufficient for benchmarking and architectural comparison, this limitation reduced the models' exposure to the extreme appearance variability required for truly robust large-scale face recognition.

A second limitation concerns the use of heuristic confidence thresholding in the similarity metric. The current implementation computes confidence using a linear transformation of the *Euclidean (L2) distance* between L2-normalized embeddings, defined as Confidence $= \max(0, (1.2 - d)/1.2) \times 100$, where a fixed distance threshold of 1.2 is assumed to separate matches from non-matches. This formulation implicitly assumes a uniform distance distribution across identities and conditions. In practice, embedding distances may vary significantly due to demographic factors, pose variation, or illumination changes, which can lead to inconsistent rejection behavior for unseen or ambiguous identities.

Finally, real-time performance is limited by the deployment pipeline itself. The system is implemented using Streamlit and OpenCV in Python, which introduces non-negligible latency due to frame capture, serialization, and model inference overhead. While sufficient for interactive demonstrations, this architecture constrains the achievable frame rate and makes the system less suitable for high-frequency or real-time surveillance scenarios when compared to optimized C++ pipelines or ONNX-based inference engines.

6.2. Future Work

To address these limitations and bridge the gap between the current prototype and a production-grade system, several directions for future work are identified. First, we plan to replace Triplet Loss with **ArcFace (Additive Angular Margin Loss)** (17). While Triplet Loss effectively enforces relative similarity constraints, ArcFace explicitly optimizes angular separation in the embedding space, encouraging tighter intra-class clustering and larger inter-class margins. Theoretical and empirical evidence suggests that this property is particularly beneficial for large-scale face recognition, where it may alleviate lookalike ambiguities observed in challenging edge cases.

Second, future work will focus on scaling the training infrastructure to support large-scale experiments. Migrating the pipeline to a high-performance computing cluster would enable full training on the MS-Celeb-1M dataset for extended training schedules exceeding 100 epochs. Such an environment would allow for a more definitive evaluation of Vision Transformer architectures, whose global attention mechanisms are hypothesized to outperform convolutional models when sufficient data and training time are available.

Finally, to mitigate inference latency and improve deployment flexibility, we propose optimizing the system for edge execution. This includes quantizing the EfficientNet backbone to INT8 precision and exporting the model to the ONNX format. These optimizations would significantly reduce memory usage and inference time, enabling the LookAlike system to operate locally on edge devices such as mobile phones or single-board computers without reliance on cloud-based processing.

A. Appendix: Implementation Details and Reproducibility

To ensure reproducibility and facilitate further experimentation, the full implementation of the *LookAlike* system—including model definitions, training scripts, preprocessing pipelines, and the interactive Streamlit application—is publicly available in a dedicated GitHub repository (18). The repository contains end-to-end code for dataset preparation, model training, embedding extraction, database indexing, and real-time inference. Due to space constraints, this appendix highlights only the most relevant implementation components that directly support the methodological choices discussed in the main paper.

A.1. Siamese Backbone Implementations

All evaluated architectures are implemented within a unified Siamese framework to ensure a fair comparison. Each backbone replaces its original classification head with a lightweight projection module that maps features into a shared 128-dimensional embedding space. The final embeddings are L2-normalized to enforce a hyperspherical geometry.

Listing 1 illustrates the EfficientNet-B0 Siamese implementation used throughout our experiments. Analogous implementations were created for ResNet-50 and ViT, differing only in backbone selection and input resolution handling. This unified design isolates architectural inductive biases while maintaining identical optimization and evaluation pipelines.

```python
class SiameseNetwork_EffNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.backbone = models.efficientnet_b0(weights=None)
        in_features = self.backbone.classifier[1].in_features
        self.backbone.classifier = nn.Sequential(
            nn.Linear(in_features, 512),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(512, 128)
        )

    def forward(self, x):
        x = self.backbone(x)
        return torch.nn.functional.normalize(x, p=2, dim=1)
```

Listing 1. EfficientNet-B0 Siamese Network with L2-Normalized Embeddings

```python
self.transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])
```

Listing 2. Image Preprocessing and Normalization Pipeline

A.2. Face Detection and Preprocessing

Prior to embedding extraction, faces are detected using an MTCNN-based detector. Detected faces are cropped, resized to 112×112, and normalized using ImageNet statistics to ensure compatibility across CNN and transformer backbones. This preprocessing (Listing 2) pipeline ensures robustness to background clutter and scale variation while preserving identity-relevant facial features.

A.3. Embedding Database Construction

To enable low-latency 1:N retrieval, embeddings for the reference gallery are precomputed offline and cached on disk (Listing 3). Each entry stores the embedding vector, image path, and identity label. The database is serialized using PyTorch's native format and loaded at runtime.This design decouples inference latency from dataset size and enables real-time retrieval even on modest hardware.

A.4. Similarity Computation and Confidence Estimation

At inference time, multiple similarity metrics are computed between the query embedding and database embeddings to support both retrieval and interpretability. Since all embeddings are L2-normalized, Euclidean distance and cosine similarity are geometrically coupled and represent equivalent measures of angular separation on the hyperspherical embedding space.

Specifically, Euclidean distance is computed using the ℓ_2 norm, while cosine similarity is reported as an auxiliary metric for user-facing interpretability. The confidence score is derived heuristically from the Euclidean distance using a fixed rejection threshold, defined as

$$\text{Confidence} = \max\left(0, \frac{1.2 - d}{1.2}\right) \times 100,$$

where d denotes the L2 distance between embeddings and a distance of 1.2 approximately corresponds to a non-match. This formulation maps smaller embedding distances to higher confidence values while assigning zero confidence to samples beyond the rejection threshold. Although cosine similarity is displayed alongside the confidence score, the confidence estimation itself is explicitly based on Euclidean distance. Given the L2 normalization constraint, both metrics induce consistent nearest-neighbor rankings, and the choice of Euclidean distance for confidence computation reflects its direct compatibility with the triplet loss optimization objective.

A.5. Dataset Acquisition Script

Dataset acquisition is automated using a lightweight script based on `kagglehub`, ensuring reproducibility with minimal manual intervention (Listing 4).

```
torch.save({
    "embeddings": final_emb,
    "paths": paths,
    "ids": ids
}, DB_FILE)
```

Listing 3. Offline Embedding Database Serialization

```
        cache_path = kagglehub.dataset_download(
    "yakhyokhuja/vggface2-112x112"
)
shutil.move(cache_path, "./dataset")
```

Listing 4. Automated Dataset Download Script

A.6. Summary

This appendix provides selected implementation details that complement the methodological discussion in the main paper. Together with the publicly available GitHub repository, these excerpts ensure transparency, reproducibility, and extensibility of the proposed *LookAlike* system.

References

[1] J. Bromley, I. Guyon, Y. LeCun, E. Säckinger, and R. Shah, "Signature verification using a "siamese" time delay neural network," in *Proceedings of the 7th International Conference on Neural Information Processing Systems*, NIPS'93, (San Francisco, CA, USA), p. 737–744, Morgan Kaufmann Publishers Inc., 1993.

[2] F. Schroff, D. Kalenichenko, and J. Philbin, "Facenet: A unified embedding for face recognition and clustering," in *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, p. 815–823, IEEE, June 2015.

[3] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," 2017.

[4] M. Tan and Q. V. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," 2020.

[5] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," 2021.

[6] F. Wang, X. Xiang, J. Cheng, and A. L. Yuille, "Normface: L2 hypersphere embedding for face verification," *Proceedings of the 25th ACM international conference on Multimedia*, 2017.

[7] D. Zeng, "Comparing contrastive and triplet loss: Variance analysis and optimization behavior," 2025.

[8] Q. Cao, L. Shen, W. Xie, O. M. Parkhi, and A. Zisserman, "Vggface2: A dataset for recognising faces across pose and age," 2018.

[9] I. de Paz Centeno, "ipazc/mtcnn: v1.0.0," Oct. 2024.

[10] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, "Joint face detection and alignment using multitask cascaded convolutional networks," *IEEE Signal Processing Letters*, vol. 23, pp. 1499–1503, Oct 2016.

[11] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 248–255, 2009.

[12] V. Yakhyokhuja, "Vggface2 preprocessed dataset," 2024.

[13] V. Yakhyokhuja, "Lfw preprocessed dataset," 2024.

[14] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2017.

[15] M. Raghu, T. Unterthiner, S. Kornblith, C. Zhang, and A. Dosovitskiy, "Do vision transformers see like convolutional neural networks?," 2022.

[16] Y. Guo, L. Zhang, Y. Hu, X. He, and J. Gao, "Ms-celeb-1m: A dataset and benchmark for large-scale face recognition," in *ECCV*, 2016.

[17] J. Deng, J. Guo, J. Yang, N. Xue, I. Kotsia, and S. Zafeiriou, "Arcface: Additive angular margin loss for deep face recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, p. 5962–5979, Oct. 2022.

[18] A. Moujtahid, "Lookalike - open-set face retrieval using deep metric learning," in *Project Report for EECS6327 (Fall 2025), Department of Electrical Engineering and Computer Science, York University*, 2025.